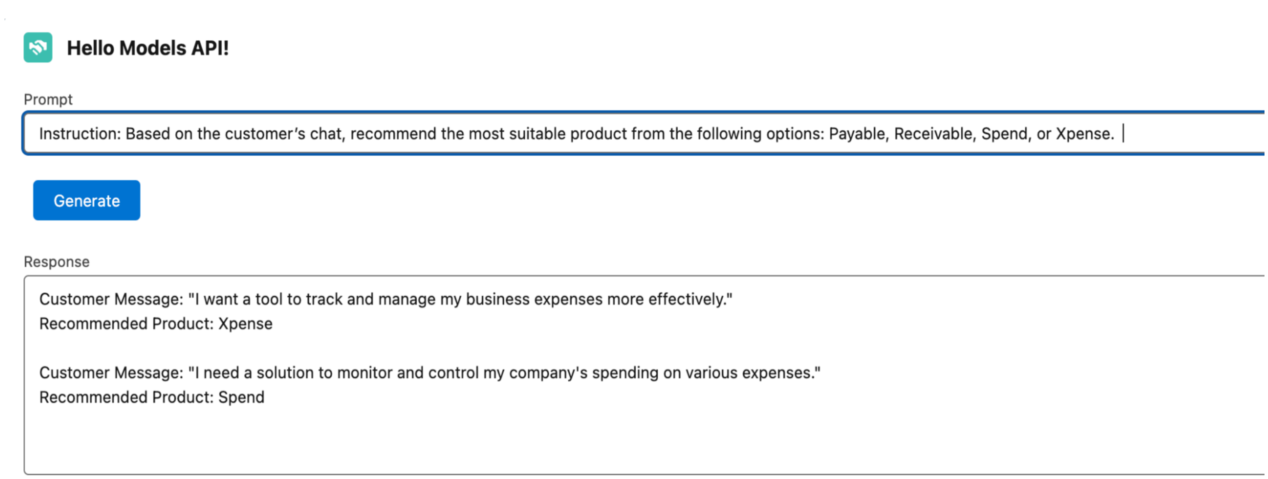
The Models API provides Apex classes and REST endpoints that connect your application to large language models (LLMs) from Salesforce partners, including Anthropic, Google, and OpenAI

1. Product Recommendations

Usage: ModelsAPI leverages historical customer interactions and product usage data to provide real-time, personalized product recommendations during customer support conversations.
How it works:

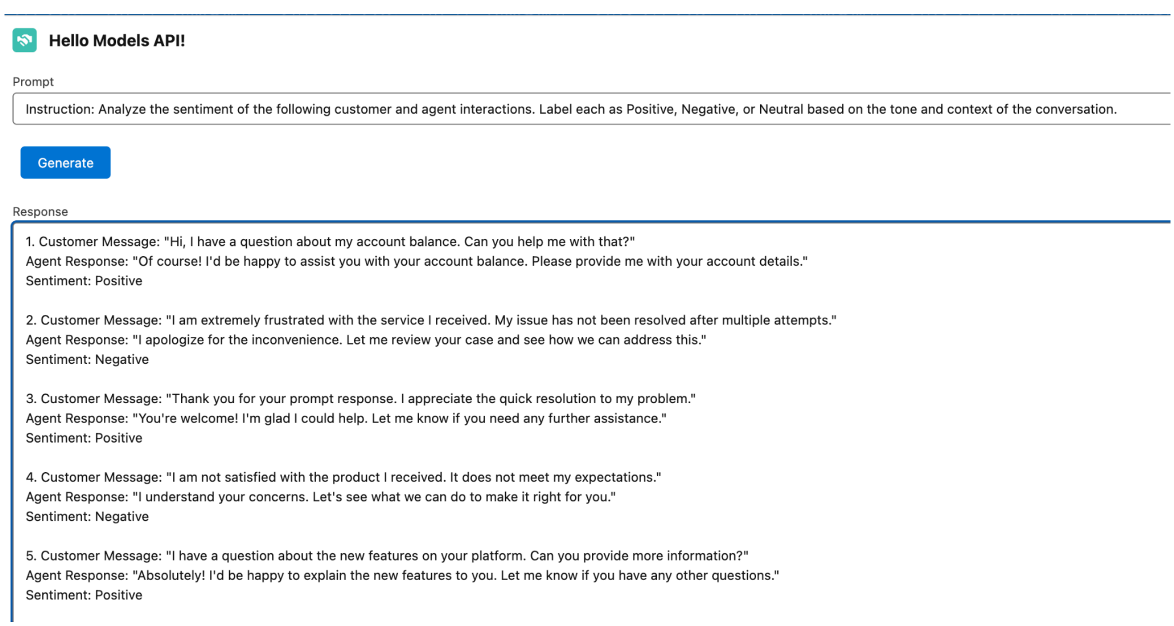
- Customer query data (e.g., chat or case details) is sent to ModelsAPI.
- The API uses a recommendation model trained on customer preferences and product metadata to suggest relevant products like Payable, Receivable, Spend, or Xpense.
- Recommendations are displayed in the agent's Service Console for seamless communication with the customer.



2. Sentiment Analysis

Usage: ModelsAPI analyzes customer messages to determine their sentiment (Positive, Negative, or Neutral) in real time. This helps agents respond appropriately and escalate issues if needed.
How it works:

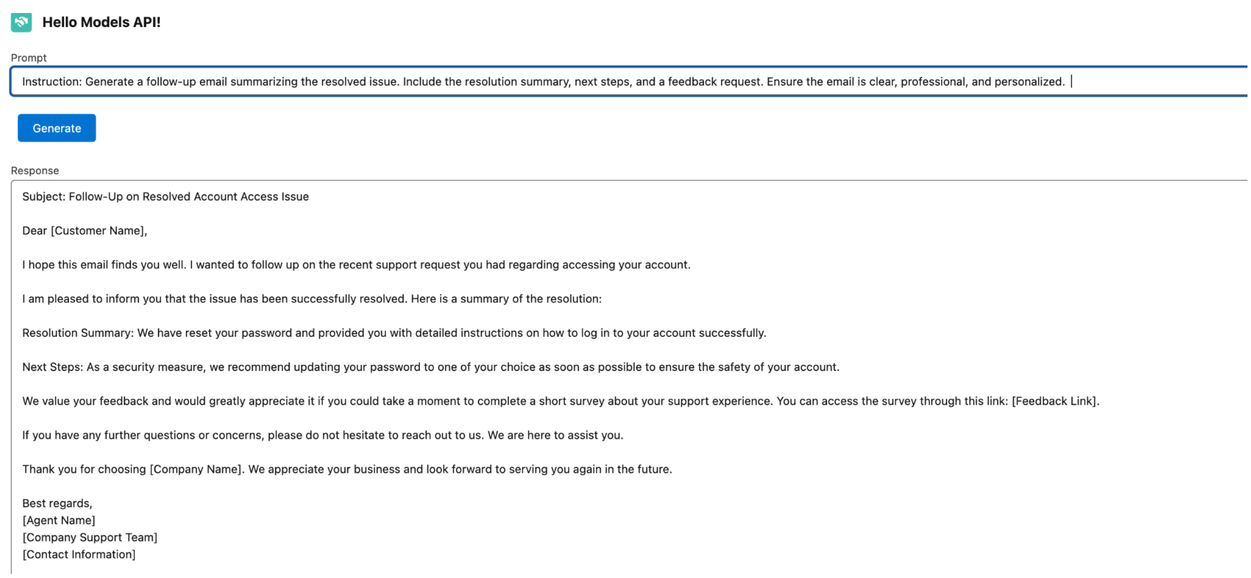
- Customer chat or email input is processed by ModelsAPI, which uses a sentiment classification model.
- The sentiment score is presented in the Service Console to guide the agent’s tone and actions.
- Negative sentiment triggers alerts for escalation, while positive sentiment could indicate high customer satisfaction.



3. Follow-Up Email Generation

Usage: ModelsAPI automates the creation of professional and personalized follow-up emails, ensuring consistency and saving agents time.
How it works:

- After case resolution, case details (issue summary, resolution actions, and next steps) are sent to ModelsAPI.
- The API uses a text generation model to draft a follow-up email.
- The email includes a summary of the resolved issue, suggested next steps, and a feedback request.
- Agents review and send the email directly from the Service Console.

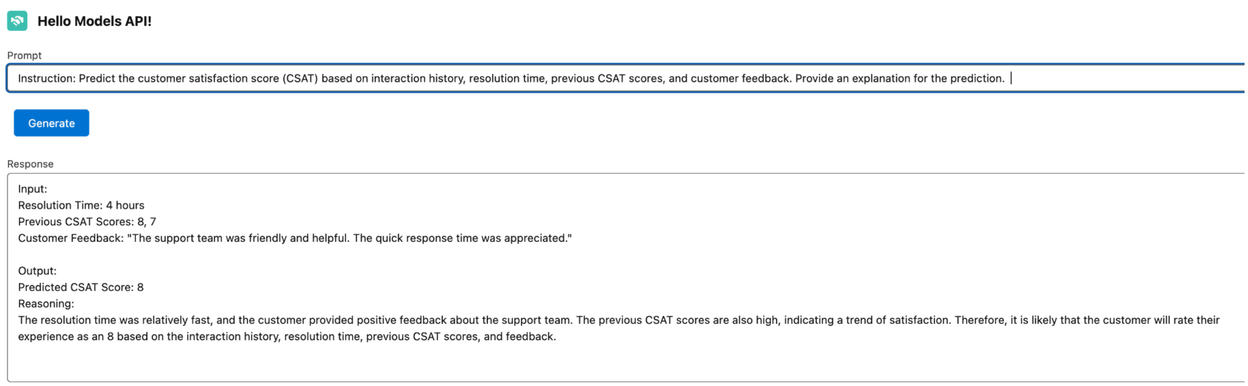


4. CSAT Score Prediction

Usage: ModelsAPI predicts Customer Satisfaction (CSAT) scores based on case interaction history, resolution times, previous CSAT scores, and customer feedback.
How it works:

- ModelsAPI uses a regression model trained on historical CSAT data and case resolution metrics.
- Case data (e.g., resolution time, previous scores, feedback) is sent to the API.

- The API predicts the CSAT score and provides reasoning (e.g., “Quick resolution with positive feedback suggests a high satisfaction score”).
- Predicted scores and explanations are available in the Service Console, enabling proactive customer experience management.




```
public class ModelsAPI2 {

   @AuraEnabled(cacheable=true)
   public static String createGeneration(String prompt) {

       // Create generations request
       aiplatform.ModelsAPI.createGenerations_Request request = new
aiplatform.ModelsAPI.createGenerations_Request();

       // Specify model
       request.modelName = 'sfdc_ai__DefaultGPT35Turbo';

       // Create request body
       aiplatform.ModelsAPI_GenerationRequest body = new
aiplatform.ModelsAPI_GenerationRequest();
       request.body = body;
```

```
        // Add prompt to body
        body.prompt = prompt;

        String modelsApiResponse = '';

        try {
            // Make request
            aiplatform.ModelsAPI modelsAPI = new aiplatform.ModelsAPI();
            aiplatform.ModelsAPI.createGenerations_Response response =
modelsAPI.createGenerations(request);

            // Add response to return value
            modelsApiResponse = response.Code200.generation.generatedText;

        // Handle error
        } catch(aiplatform.ModelsAPI.createGenerations_ResponseException e) {
            System.debug('Response code: ' + e.responseCode);
            System.debug('The following exception occurred: ' + e);

            // Add error to the return value
            modelsApiResponse = 'Unable to get a valid response. Error code: ' +
e.responseCode;
        }

        // Return response
        return modelsApiResponse;
    }
}
```

```
import { LightningElement } from 'lwc';
import createGeneration from '@salesforce/apex/ModelsAPI2.createGeneration';

export default class HelloWorld extends LightningElement {
   prompt = 'Generate a welcome email for the new developer on the team, Jane Doe.';
   response = '';

   handlePromptChange(event) {
       this.prompt = event.target.value;
```

```
    }

    handleClick(event) {
        this.response = 'Working…';
        createGeneration({ prompt: this.prompt })
        .then(result => {
            this.response = result;
            this.error = undefined;
        })
        .catch(error => {
            this.response = 'Error';
            this.error = error;
        })
    }
}
```

```
<template>
    <lightning-card title="Hello Models API!" icon-name="custom:custom14">
        <div class="slds-m-around_medium">
            <lightning-input label="Prompt" value={prompt}
onchange={handlePromptChange}>
                ></lightning-input>
            <br/>
            <lightning-button variant="brand" label="Generate" title="Generate
response" onclick={handleClick} class="slds-m-left_x-small"></lightning-button>
            <br/><br/>
            <lightning-textarea label="Response" value={response} readonly>
</lightning-textarea>
        </div>
    </lightning-card>
</template>
```

```
<?xml version="1.0"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>61.0</apiVersion>
    <isExposed>true</isExposed>
    <targets>
        <target>lightning__RecordPage</target>
        <target>lightning__HomePage</target>
    </targets>
</LightningComponentBundle>
```